

**TRƯỜNG ĐẠI HỌC KHOA HỌC
KHOA CÔNG NGHỆ THÔNG TIN**

**TRƯỜNG ĐẠI HỌC KHOA HỌC HUẾ
HUE UNIVERSITY OF SCIENCES**



ĐỀ TÀI
HỆ THỐNG ĐO NHỊP TIM VÀ GỬI DỮ LIỆU QUA
ESP32

Học phần : Phát triển ứng dụng IoT - Nhóm 3 - TIN4024.003
Sinh viên thực hiện : Nguyễn Thắng
Mã sinh viên : 22T1020428
Giảng viên hướng dẫn : Võ Việt Dũng

Huế, 06/2026

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Giải thích
IoT	Internet of Things – Internet Vạn vật
ESP32	Tên vi điều khiển của hãng Espressif, tích hợp WiFi và Bluetooth
BPM	Beats Per Minute – Nhịp tim tính theo số lần đập mỗi phút
SpO2	Saturation of Peripheral Oxygen – Độ bão hòa oxy máu ngoại vi
PPG	Photoplethysmography – Kỹ thuật đo xung quang học
IR	Infrared – Ánh sáng hồng ngoại
I2C	Inter-Integrated Circuit – Chuẩn giao tiếp nối tiếp 2 dây
LCD	Liquid Crystal Display – Màn hình tinh thể lỏng
MQTT	Message Queuing Telemetry Transport – Giao thức truyền thông nhẹ
HTTP	Hypertext Transfer Protocol – Giao thức truyền tải siêu văn bản

Mục lục

Danh mục từ viết tắt

Mở đầu	1
1 CƠ SỞ LÝ THUYẾT	3
1.1 Tổng quan về Internet Vạn vật (IoT)	3
1.2 IoT trong y tế và giám sát sức khỏe	3
1.3 Các nền tảng IoT phổ biến	4
1.4 Kỹ thuật đo nhịp tim bằng PPG	4
1.5 Tính toán BPM từ tín hiệu PPG	5
1.6 Tính toán SpO2 bằng phương pháp Beer-Lambert	5
1.7 Giao tiếp I2C	6
2 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	7
2.1 Yêu cầu hệ thống	7
2.2 Giới thiệu các thành phần phần cứng	7
2.2.1 Vi điều khiển ESP32 DevKit V1	7
2.2.2 Cảm biến MAX30100	8
2.2.3 Màn hình LCD 1602 I2C	9
2.2.4 Hệ thống LED chỉ thị	9
2.3 Kiến trúc tổng thể hệ thống	10
2.4 Sơ đồ kết nối phần cứng	10
2.5 Thiết kế phần mềm	11
2.5.1 Cấu trúc dự án	11
2.5.2 Thiết kế module mô phỏng MAX30100_Sim.h	12
2.5.3 Cấu hình Blynk Datastream	12
3 TRIỂN KHAI VÀ ĐÁNH GIÁ KẾT QUẢ	14
3.1 Môi trường và quy trình triển khai	14
3.1.1 Công cụ phát triển	14
3.1.2 Quy trình build và chạy mô phỏng	14
3.2 Kết quả mô phỏng Wokwi	14
3.2.1 Giai đoạn warm-up	14
3.2.2 Trạng thái hoạt động bình thường	15
3.2.3 Trạng thái cảnh báo	15
3.3 Kết quả trên Blynk Dashboard	16
3.4 Phân tích và đánh giá	17
3.4.1 Phân tích kết quả theo các tình huống	17
3.4.2 Đánh giá ưu điểm và hạn chế	17
3.5 Hướng phát triển	17
Kết luận	19
Tài liệu tham khảo	20

MỞ ĐẦU

1. Lý do chọn đề tài

Trong bối cảnh bùng nổ của cuộc cách mạng công nghiệp 4.0, Internet Vạn vật (IoT) đang dần thâm nhập vào mọi lĩnh vực của cuộc sống, đặc biệt là y tế và chăm sóc sức khỏe. Các thiết bị giám sát sức khỏe thông minh có khả năng đo đạc, phân tích và truyền dữ liệu sinh lý theo thời gian thực đang trở thành xu hướng tất yếu trong nền y học hiện đại.

Nhịp tim (Heart Rate) và độ bão hòa oxy trong máu (SpO2) là hai chỉ số sinh lý quan trọng hàng đầu. Nhịp tim bình thường của người trưởng thành dao động từ 60 đến 100 lần/phút; SpO2 bình thường từ 95% đến 100%. Bất kỳ sự sai lệch nào khỏi ngưỡng này đều có thể là dấu hiệu cảnh báo về các vấn đề tim mạch, hô hấp hoặc tuần hoàn máu.

Việc xây dựng một hệ thống giám sát nhịp tim và SpO2 nhỏ gọn, chi phí thấp, có khả năng gửi dữ liệu lên nền tảng đám mây để theo dõi từ xa là một bài toán thiết thực. Đề tài này ứng dụng vi điều khiển ESP32 kết hợp cảm biến quang học MAX30100 để giải quyết bài toán đó, đồng thời minh họa rõ quy trình phát triển một ứng dụng IoT hoàn chỉnh từ phần cứng đến phần mềm đám mây.

2. Mục tiêu đề tài

- Tìm hiểu nguyên lý đo nhịp tim và SpO2 bằng phương pháp PPG (Photoplethysmography).
- Thiết kế hệ thống phần cứng sử dụng ESP32 và MAX30100 với màn hình LCD và các LED chỉ thị.
- Xây dựng phần mềm nhúng trên ESP32 để đọc, xử lý và hiển thị dữ liệu.
- Tích hợp nền tảng đám mây Blynk để giám sát dữ liệu từ xa theo thời gian thực.
- Mô phỏng hệ thống trên Wokwi nhằm kiểm chứng thiết kế trước khi triển khai thực tế.

3. Phạm vi và giới hạn

Đề tài tập trung vào giai đoạn thiết kế, lập trình và mô phỏng. Do công cụ mô phỏng Wokwi không hỗ trợ cảm biến MAX30100, tôi đã xây dựng module mô phỏng vật lý MAX30100_Sim.h để tái tạo tín hiệu PPG theo đúng nguyên lý quang học, bao gồm nhiễu tín hiệu và hiệu ứng chuyển động (motion artifact). Hệ thống thực tế có thể được triển khai bằng cách thay thế module này bằng thư viện cảm biến chính thức.

4. Cấu trúc bài tiểu luận

Bài tiểu luận được cấu trúc thành các chương và phần chính như sau:

- **Phần Mở đầu:** Trình bày lý do chọn đề tài, mục tiêu nghiên cứu, phạm vi và giới hạn của đề tài cùng cấu trúc tổng quát của bài tiểu luận[cite: 330, 752].
- **Chương I - Cơ sở lý thuyết:** Tổng quan về Internet Vạn vật (IoT), ứng dụng IoT trong y tế, kỹ thuật đo PPG, các phương pháp tính toán chỉ số sinh lý (BPM, SpO2) và chuẩn giao tiếp I2C.

- **Chương II - Phân tích và thiết kế hệ thống:** Xác định yêu cầu hệ thống; giới thiệu các thành phần phần cứng như ESP32, MAX30100, LCD 1602; thiết kế kiến trúc tổng thể, sơ đồ kết nối và cấu trúc phần mềm (bao gồm module mô phỏng MAX30100_Sim).
- **Chương III - Triển khai và đánh giá kết quả:** Mô tả môi trường phát triển, quy trình thực hiện, kết quả mô phỏng trên Wokwi, dữ liệu thực tế trên Blynk Dashboard và các đánh giá về ưu nhược điểm của hệ thống.
- **Phần Kết luận:** Tổng kết các kết quả đã đạt được, bài học kinh nghiệm thu được và đề xuất các hướng phát triển mở rộng cho đề tài trong tương lai.

1. CƠ SỞ LÝ THUYẾT

1.1. Tổng quan về Internet Vạn vật (IoT)

Internet Vạn vật (IoT) là mạng lưới các thiết bị vật lý được trang bị cảm biến, phần mềm và kết nối mạng, cho phép chúng thu thập và trao đổi dữ liệu với nhau và với hệ thống trung tâm. Một hệ thống IoT điển hình bao gồm bốn lớp chức năng:

1. **Lớp cảm nhận (Perception Layer):** Các cảm biến thu thập dữ liệu từ thế giới vật lý như nhiệt độ, ánh sáng, chuyển động, nhịp tim, v.v.
2. **Lớp mạng (Network Layer):** Truyền dữ liệu qua các giao thức không dây (WiFi, Bluetooth, Zigbee, LoRa) hoặc có dây.
3. **Lớp xử lý (Processing Layer):** Lưu trữ, phân tích và xử lý dữ liệu trên nền tảng đám mây hoặc edge computing.
4. **Lớp ứng dụng (Application Layer):** Giao diện người dùng, dashboard theo dõi, hệ thống cảnh báo và tự động hóa.

1.2. IoT trong y tế và giám sát sức khỏe

Ứng dụng IoT trong y tế (Healthcare IoT hay MIoT) đang tạo ra cuộc cách mạng trong cách theo dõi sức khỏe bệnh nhân. Các thiết bị đeo tay thông minh (wearables) như đồng hồ Apple Watch, Fitbit hay Garmin đều tích hợp cảm biến quang học để đo nhịp tim liên tục. Trong môi trường bệnh viện, các thiết bị giám sát bệnh nhân từ xa (Remote Patient Monitoring – RPM) giúp bác sĩ theo dõi tình trạng bệnh nhân mà không cần tiếp xúc trực tiếp, đặc biệt hữu ích trong bối cảnh đại dịch hay bệnh nhân ở vùng sâu, vùng xa.

Hệ thống trong đề tài này thuộc loại thiết bị giám sát sức khỏe cơ bản, tập trung vào hai chỉ số quan trọng nhất: nhịp tim (BPM) và nồng độ oxy trong máu (SpO2). Ứng dụng thực tiễn bao gồm:

- Theo dõi sức khỏe người cao tuổi tại nhà, cảnh báo người thân khi có bất thường.
- Giám sát bệnh nhân tim mạch trong quá trình phục hồi sau phẫu thuật.
- Hỗ trợ phát hiện sớm các dấu hiệu suy hô hấp, rối loạn nhịp tim.

1.3. Các nền tảng IoT phổ biến

Bảng 2: So sánh các nền tảng IoT phổ biến

Nền tảng	Ưu điểm	Nhược điểm	Phù hợp với
Blynk	Dễ dùng, dashboard đẹp, free tier	Giới hạn datastream miễn phí	Prototype, học tập
ThingSpeak	Tích hợp MATLAB, vẽ đồ thị tốt	Giới hạn 4 kênh miễn phí	Phân tích dữ liệu
Telegram	Miễn phí hoàn toàn, alert tức thì	Không có dashboard	Cảnh báo đơn giản
AWS IoT	Scalable, enterprise-grade	Phức tạp, tốn phí	Sản phẩm thương mại
Firebase	Realtime database, free tier	Không chuyên IoT	App mobile kết hợp

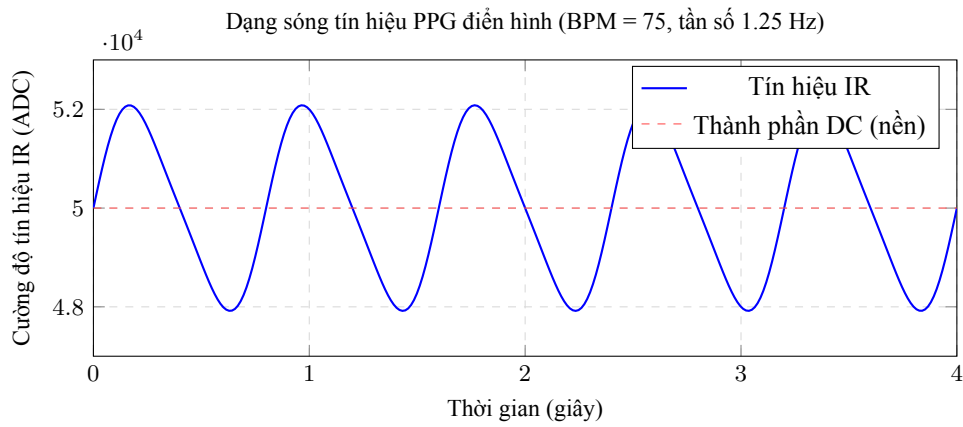
Cá nhân chọn **Blynk** vì tính đơn giản trong thiết lập, giao diện dashboard trực quan, hỗ trợ tốt cho ESP32 và có thư viện Arduino chính thức với tài liệu đầy đủ, phù hợp với mục tiêu học tập của môn học.

1.4. Kỹ thuật đo nhịp tim bằng PPG

Photoplethysmography (PPG) là kỹ thuật đo thể tích máu thay đổi trong mao mạch bằng cách phân tích sự hấp thụ ánh sáng. Đây là nguyên lý nền tảng của tất cả các thiết bị đo SpO2 và nhịp tim không xâm lấn hiện đại.

Nguyên lý hoạt động cụ thể như sau:

1. LED hồng ngoại (880nm) và LED đỏ (660nm) chiếu ánh sáng xuyên qua hoặc phản xạ trên đầu ngón tay.
2. Mỗi khi tim đập, một lượng máu mới được bơm vào mao mạch ngón tay, làm tăng thể tích mô → tăng hấp thụ ánh sáng → photodetector nhận được ít ánh sáng hơn → tín hiệu ADC giảm.
3. Giữa hai nhịp đập (tâm trương), lượng máu giảm → hấp thụ ít hơn → photodetector nhận nhiều ánh sáng hơn → tín hiệu ADC tăng.
4. Sự dao động tuần hoàn này tạo ra tín hiệu PPG dạng sóng, tần số chính bằng nhịp tim.



Hình 1: Dạng sóng PPG điển hình: phần DC (nền) và phần AC (dao động theo nhịp tim)

Tín hiệu PPG gồm hai thành phần:

- **Thành phần DC:** Giá trị nền không đổi, đại diện cho mô không có máu (xương, da, cơ). Trong hình trên, đường đứt nét DC = 50000 ADC.
- **Thành phần AC:** Phần dao động theo nhịp tim, đại diện cho lượng máu thay đổi. Đây là thông tin cần khai thác để tính BPM và SpO2.

1.5. Tính toán BPM từ tín hiệu PPG

Từ tín hiệu IR thu được, thuật toán tính BPM theo các bước:

1. Tách thành phần AC bằng cách trừ giá trị trung bình: $AC_{IR}(t) = IR(t) - \overline{IR}$
2. Phát hiện đỉnh sóng (peak detection) trong tín hiệu AC bằng thuật toán ngưỡng.
3. Đếm số đỉnh trong cửa sổ thời gian T giây.
4. Tính BPM: $BPM = \frac{N_{peaks}}{T} \times 60$

Trong module mô phỏng, thuật toán zero-crossing được dùng thay thế cho peak detection do đơn giản hơn và ít bị ảnh hưởng bởi nhiễu:

$$BPM = N_{crossings} \times \frac{60}{T_{buffer}}$$

1.6. Tính toán SpO2 bằng phương pháp Beer-Lambert

SpO2 được tính dựa trên sự khác biệt hấp thụ ánh sáng của hemoglobin có oxy (HbO₂) và không có oxy (Hb) ở hai bước sóng. HbO₂ hấp thụ nhiều ánh sáng hồng ngoại hơn, trong khi Hb hấp thụ nhiều ánh sáng đỏ hơn. Tỷ lệ này cho phép tính ra phần trăm oxy trong máu.

Bước 1 – Tính hệ số R (Ratio of Ratios):

$$R = \frac{AC_{RED}/DC_{RED}}{AC_{IR}/DC_{IR}}$$

Bước 2 – Tính SpO2 từ R:

$$SpO_2 = 110 - 25 \times R$$

Đây là xấp xỉ tuyến tính của đường cong Beer-Lambert, hợp lệ trong khoảng $SpO_2 \in [70\%, 100\%]$ [?].

Bảng 3: Bảng tra giá trị R và SpO2 tương ứng

Giá trị R	SpO2 (%)	Tình trạng lâm sàng
0.40	100	Rất tốt
0.52	97	Bình thường
0.60	95	Ngưỡng thấp
0.80	90	Cần theo dõi
1.00	85	Nguy hiểm
1.20	80	Rất nguy hiểm, cần can thiệp

1.7. Giao tiếp I2C

I2C (Inter-Integrated Circuit) là chuẩn giao tiếp nối tiếp đồng bộ 2 dây do Philips (nay là NXP) phát triển, cho phép nhiều thiết bị ngoại vi (slave) cùng chia sẻ một bus với vi điều khiển (master). Hai dây tín hiệu gồm:

- **SDA (Serial Data):** Đường dữ liệu hai chiều.
- **SCL (Serial Clock):** Đường xung nhịp, do master điều khiển.

Mỗi thiết bị trên bus I2C có một địa chỉ 7-bit duy nhất. Trong hệ thống này, MAX30100 sử dụng địa chỉ 0x57 và LCD 1602 sử dụng địa chỉ 0x27, do đó cả hai có thể dùng chung bus I2C trên GPIO21 (SDA) và GPIO22 (SCL) của ESP32 mà không xung đột.

2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Yêu cầu hệ thống

Trước khi thiết kế, đã xác định các yêu cầu chức năng và phi chức năng của hệ thống:

Yêu cầu chức năng:

- Đo nhịp tim (BPM) và nồng độ oxy trong máu (SpO2) liên tục.
- Hiển thị kết quả trực tiếp trên LCD 1602 tại thiết bị.
- Gửi dữ liệu lên nền tảng đám mây Blynk mỗi 5 giây.
- Cảnh báo bằng LED khi phát hiện nhịp tim hoặc SpO2 bất thường.
- Chỉ thị trạng thái kết nối WiFi/Blynk.

Yêu cầu phi chức năng:

- Chi phí phần cứng thấp, linh kiện phổ biến, dễ mua.
- Mô phỏng được trên Wokwi để kiểm thử không cần phần cứng.
- Code có thể chuyển sang phần cứng thực với thay đổi tối thiểu.

2.2. Giới thiệu các thành phần phần cứng

2.2.1. Vi điều khiển ESP32 DevKit V1

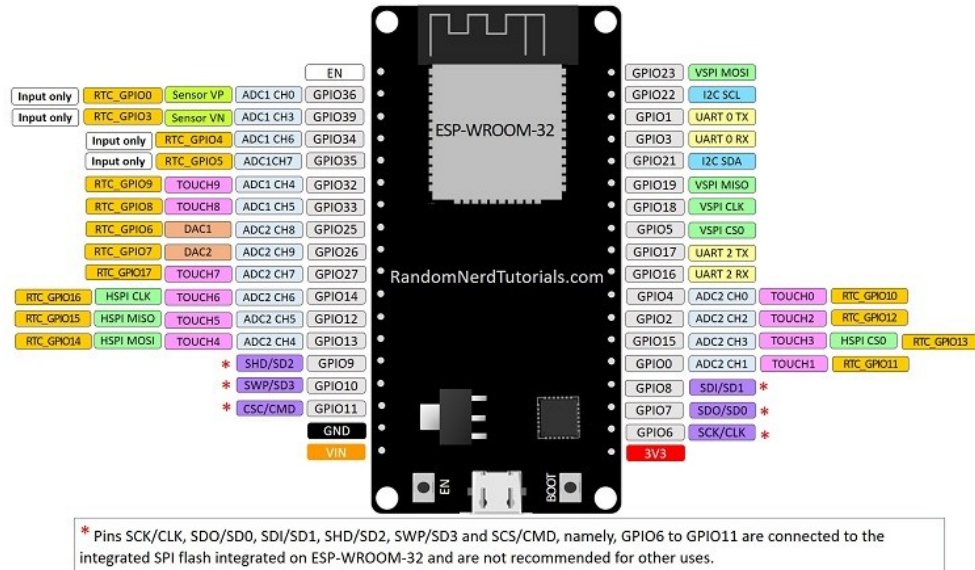
ESP32 là vi điều khiển do công ty Espressif Systems (Trung Quốc) phát triển, ra mắt năm 2016 và nhanh chóng trở thành nền tảng IoT phổ biến nhờ tích hợp WiFi 802.11 b/g/n và Bluetooth 4.2/BLE trong một chip duy nhất với giá thành rất thấp (khoảng 5–10 USD/module).

Bảng 4: Thông số kỹ thuật ESP32 DevKit V1

Thông số	Giá trị
CPU	Xtensa LX6 dual-core, 240 MHz
RAM	520 KB SRAM
Flash	4 MB
WiFi	802.11 b/g/n, 2.4 GHz
Bluetooth	Classic BT 4.2 + BLE
GPIO	30 chân (15 mỗi bên)
ADC	12-bit, 18 kênh
I2C	2 bus (GPIO21/22 mặc định)
Nguồn	3.3V (GPIO), 5V (USB)
Điện năng	240 mA (peak WiFi)

ESP32 DEVKIT V1 – DOIT

version with 36 GPIOs



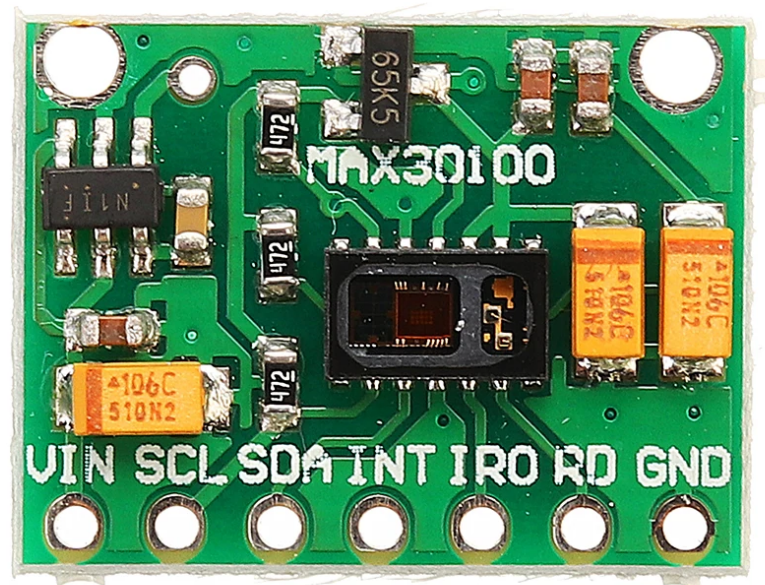
Hình 2: Sơ đồ chân ESP32 DevKit V1

2.2.2. Cảm biến MAX30100

MAX30100 là module Pulse Oximeter do Maxim Integrated (nay thuộc Analog Devices) sản xuất, tích hợp hai đèn LED (đỏ 660nm và hồng ngoại 880nm), photodetector độ nhạy cao và mạch xử lý tín hiệu analog trên một chip. Đây là cảm biến chuyên dụng cho ứng dụng y tế đeo tay.

Bảng 5: Thông số kỹ thuật MAX30100

Thông số	Giá trị
Giao tiếp	I2C, địa chỉ 0x57
Điện áp hoạt động	1.8V – 3.3V
LED đỏ	660 nm
LED hồng ngoại	880 nm
ADC	16-bit
Tần số lấy mẫu	50 – 1000 Hz (cấu hình được)
Dòng LED tối đa	50 mA
Kích thước module	14mm × 17mm



Hình 3: Module cảm biến MAX30100

2.2.3. Màn hình LCD 1602 I2C

LCD 1602 là màn hình tinh thể lỏng 16 ký tự $\times$ 2 hàng. Module sử dụng chip chuyển đổi PCF8574 gắn phía sau, cho phép giao tiếp I2C thay vì parallel 8-bit, giảm từ 6–8 dây xuống còn 4 dây (VCC, GND, SDA, SCL).

LCD chia sẻ bus I2C với MAX30100 mà không xung đột nhờ địa chỉ khác nhau: LCD dùng 0x27, MAX30100 dùng 0x57.

2.2.4. Hệ thống LED chỉ thị

Bảng 6: Chức năng các LED chỉ thị

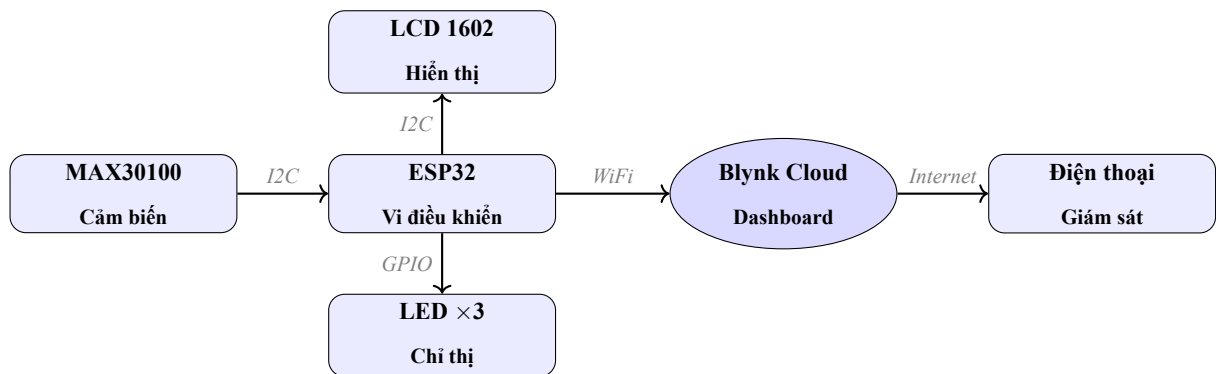
LED	Màu	GPIO	Chức năng
Heart Beat	Đỏ	2	Nhấp nháy theo tần số BPM
Blynk OK	Xanh	4	Sáng liên tục khi kết nối Blynk thành công
Warning	Vàng	5	Sáng khi BPM < 60, > 100 hoặc SpO2 < 95%

Điện trở hạn dòng 220Ω cho mỗi LED được tính như sau:

$$R = \frac{V_{GPIO} - V_{LED}}{I_{LED}} = \frac{3,3 \text{ V} - 2,0 \text{ V}}{0,02 \text{ A}} = 65 \Omega \rightarrow \text{chọn } 220 \Omega \text{ (an toàn hơn)}$$

2.3. Kiến trúc tổng thể hệ thống

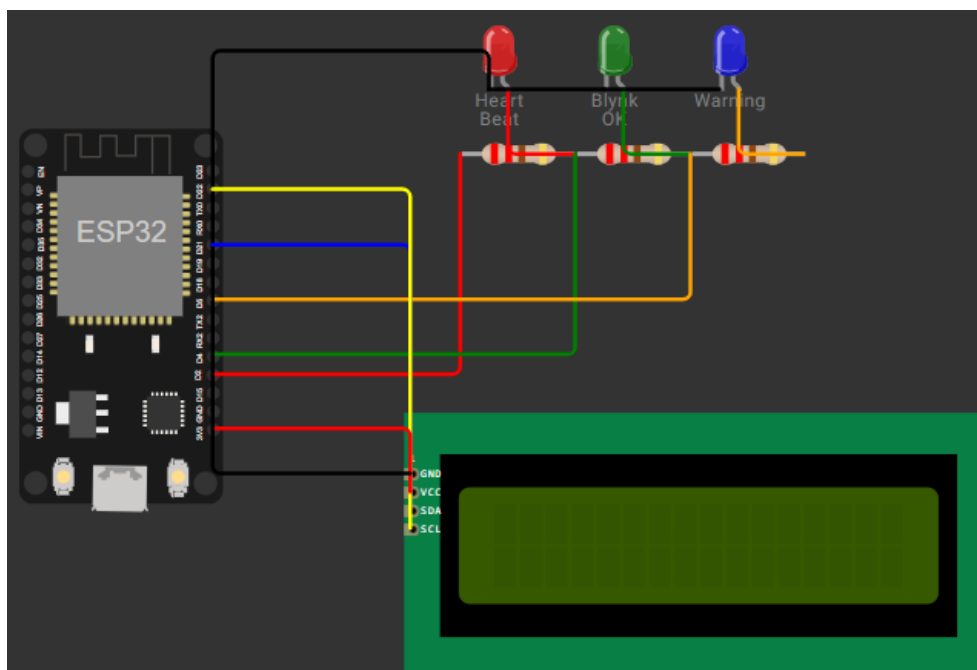
Dựa trên các yêu cầu đã đặt ra, hệ thống được thiết kế theo mô hình mạng hình sao (Hub-and-Spoke), trong đó vi điều khiển ESP32 đóng vai trò là bộ não xử lý trung tâm. Các thiết bị ngoại vi (cảm biến, màn hình, LED) được kết nối xung quanh ESP32 thông qua các chuẩn giao tiếp nội bộ (I2C, GPIO), trong khi luồng dữ liệu đầu ra được đồng bộ lên đám mây qua mạng WiFi. Hình 4 mô tả chi tiết sự tương tác giữa các lớp vật lý và lớp đám mây trong hệ thống.



Hình 4: Kiến trúc tổng thể hệ thống đo nhịp tim IoT

2.4. Sơ đồ kết nối phần cứng

Từ kiến trúc tổng thể, sơ đồ mạch điện chi tiết được thiết kế và triển khai trực quan trên môi trường mô phỏng Wokwi (Hình 5). Để đảm bảo tính chính xác và thuận tiện cho việc chuyển đổi từ mô phỏng sang lắp ráp phần cứng thực tế, toàn bộ quy tắc đi dây (pinout) giữa các linh kiện được tổng hợp chi tiết trong Bảng 7.



Hình 5: Sơ đồ mạch mô phỏng trên Wokwi

Bảng 7: Bảng kết nối toàn bộ hệ thống

Thiết bị	Chân thiết bị	Chân ESP32	Màu dây
MAX30100	VIN	3.3V	Đỏ
	GND	GND	Đen
	SDA	GPIO21	Xanh lam
	SCL	GPIO22	Vàng
LCD 1602 I2C	VCC	3.3V	Đỏ
	GND	GND	Đen
	SDA	GPIO21	Xanh lam
	SCL	GPIO22	Vàng
LED Heart Beat (đỏ)	Anode (+) qua 220Ω	GPIO2	Đỏ
LED Blynk OK (xanh)	Anode (+) qua 220Ω	GPIO4	Xanh lá
LED Warning (vàng)	Anode (+) qua 220Ω	GPIO5	Cam

2.5. Thiết kế phần mềm

2.5.1. Cấu trúc dự án

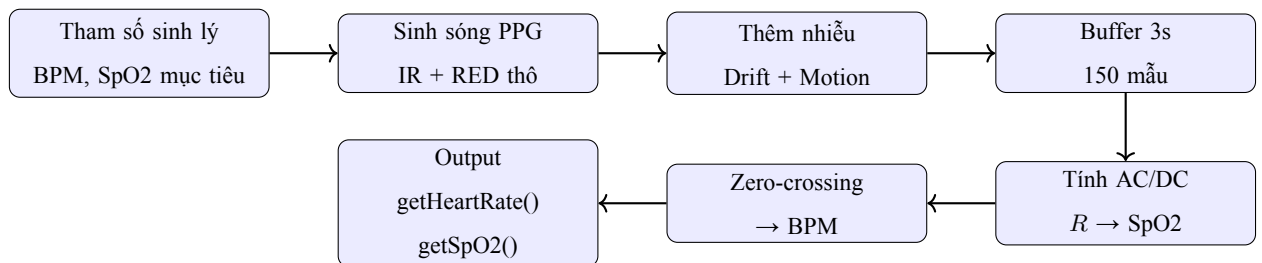
Để đảm bảo tính tổ chức, dễ bảo trì và thuận tiện cho việc gỡ lỗi (debug), toàn bộ mã nguồn của hệ thống được phát triển trên môi trường PlatformIO và chia nhỏ thành các tệp tin có vai trò chuyên biệt. Bảng 8 tóm tắt cấu trúc thư mục và chức năng của từng tệp tin cốt lõi trong dự án:

Bảng 8: Cấu trúc file dự án PlatformIO

File	Vai trò	Vị trí
main.cpp	Chương trình chính (setup + loop)	src/
MAX30100_Sim.h	Module mô phỏng cảm biến vật lý	Thư mục gốc
platformio.ini	Cấu hình board + thư viện	Thư mục gốc
wokwi.toml	Liên kết firmware với Wokwi	Thư mục gốc
diagram.json	Sơ đồ mạch điện Wokwi	Thư mục gốc

2.5.2. Thiết kế module mô phỏng MAX30100_Sim.h

Do Wokwi không hỗ trợ cảm biến MAX30100, tôi xây dựng module MAX30100_Sim.h mô phỏng đầy đủ pipeline vật lý theo chiều ngược lại: thay vì đọc từ sensor, module sinh tín hiệu IR/RED từ tham số sinh lý định sẵn, rồi tính ngược lại BPM và SpO2 bằng đúng công thức vật lý.



Hình 6: Pipeline xử lý trong module MAX30100_Sim.h

Module có API giống hệt thư viện MAX30100 thật, cho phép chuyển sang phần cứng thực chỉ bằng cách đổi `#include`:

Bảng 9: So sánh API mô phỏng và sensor thật

Hàm	MAX30100_Sim.h	MAX30100lib (thật)
Khởi tạo	<code>pox.begin()</code>	<code>pox.begin()</code>
Cập nhật	<code>pox.update()</code>	<code>pox.update()</code>
Lấy nhịp tim	<code>pox.getHeartRate()</code>	<code>pox.getHeartRate()</code>
Lấy SpO2	<code>pox.getSpO2()</code>	<code>pox.getSpO2()</code>
Trạng thái	<code>pox.isReady()</code>	Không có (thêm mới)

2.5.3. Cấu hình Blynk Datastream

Để đồng bộ dữ liệu giữa vi điều khiển ESP32 và nền tảng đám mây, hệ thống sử dụng các chân ảo (Virtual Pins) trên Blynk. Việc phân bổ chân ảo giúp định tuyến chính xác các loại dữ liệu (số nguyên, số thực, chuỗi ký tự) lên các Widget tương ứng trên giao diện người dùng. Chi tiết cấu hình các luồng dữ liệu (Datastream) được thể hiện trong Bảng 10:

Bảng 10: Cấu hình Virtual Pin trên hệ thống Blynk

Virtual Pin	Tên	Kiểu	Phạm vi	Widget
V0	Heart Rate	Integer	40 – 200 BPM	Gauge
V1	SpO2	Double	80 – 100 %	Gauge
V2	Status	String	—	Label
V3	Timestamp	String	—	Label
V5	Reset	Integer	0 – 1	Button (PUSH)

Bên cạnh việc gửi dữ liệu lên, hệ thống cũng sử dụng chân V5 dưới dạng nút nhấn (Push Button) để cho phép người dùng gửi lệnh điều khiển từ xa về ESP32 nhằm reset lại giao diện cục bộ khi cần thiết.

3. TRIỂN KHAI VÀ ĐÁNH GIÁ KẾT QUẢ

3.1. Môi trường và quy trình triển khai

3.1.1. Công cụ phát triển

Hệ thống được phát triển và kiểm thử trên bộ công cụ sau:

- **VS Code + PlatformIO:** IDE và build system chính. PlatformIO tự động tải thư viện, quản lý dependencies và build firmware `.bin`.
- **Wokwi for VS Code:** Extension mô phỏng phần cứng, đọc trực tiếp `firmware.bin` từ PlatformIO, hỗ trợ ESP32, LCD, LED, WiFi ảo.
- **Blynk Cloud:** Nền tảng IoT miễn phí, cung cấp dashboard web và app mobile để hiển thị dữ liệu realtime.

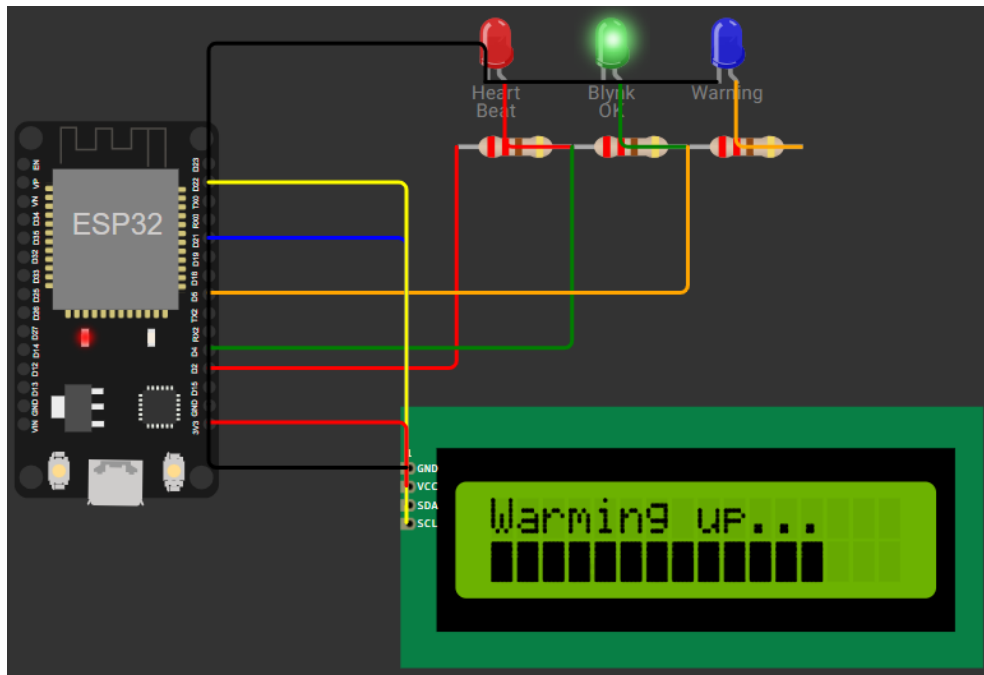
3.1.2. Quy trình build và chạy mô phỏng

1. Viết code trong `src/main.cpp`, cấu hình thư viện trong `platformio.ini`.
2. Build firmware: **Ctrl+Shift+B** → PlatformIO biên dịch.
3. Khởi động mô phỏng: **Ctrl+Shift+P** → *Wokwi: Start Simulator*.
4. Wokwi đọc `firmware.bin` + `diagram.json` và chạy mô phỏng theo thời gian thực.
5. Kiểm tra Serial Monitor (115200 baud) và theo dõi Blynk dashboard.

3.2. Kết quả mô phỏng Wokwi

3.2.1. Giai đoạn warm-up

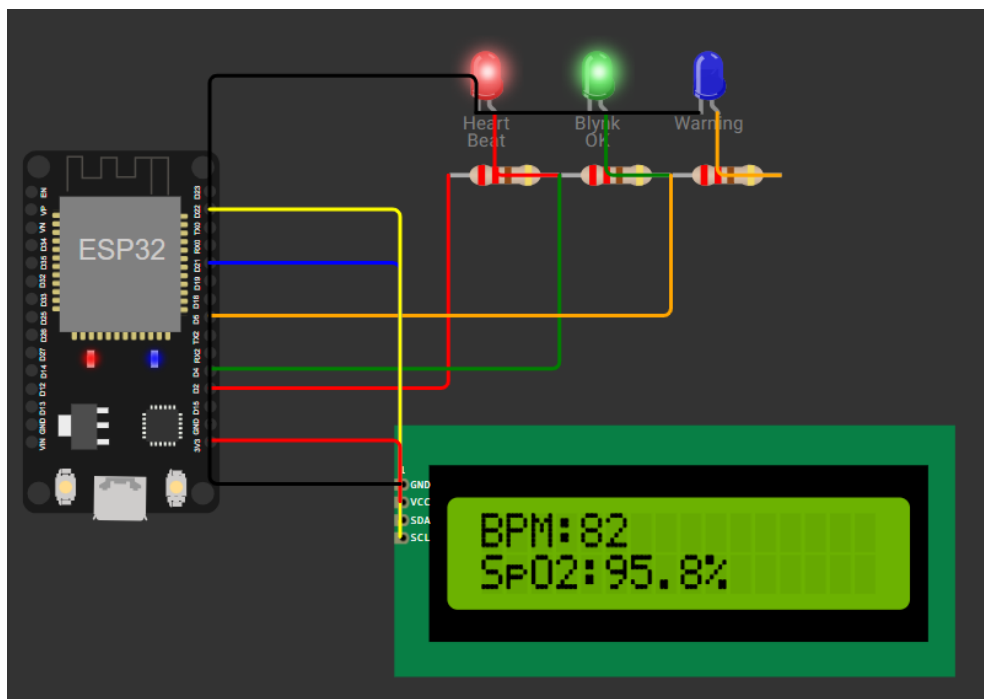
Trong 3 giây đầu sau khi khởi động, module `MAX30100_Sim.h` chưa có đủ dữ liệu trong buffer để tính BPM và SpO2. LCD hiển thị thanh tiến trình thay vì số liệu:



Hình 7: Màn hình LCD trong giai đoạn warm-up (3 giây đầu)

3.2.2. *Trạng thái hoạt động bình thường*

Sau khi warm-up, hệ thống hoạt động ổn định với các giá trị sinh lý bình thường:

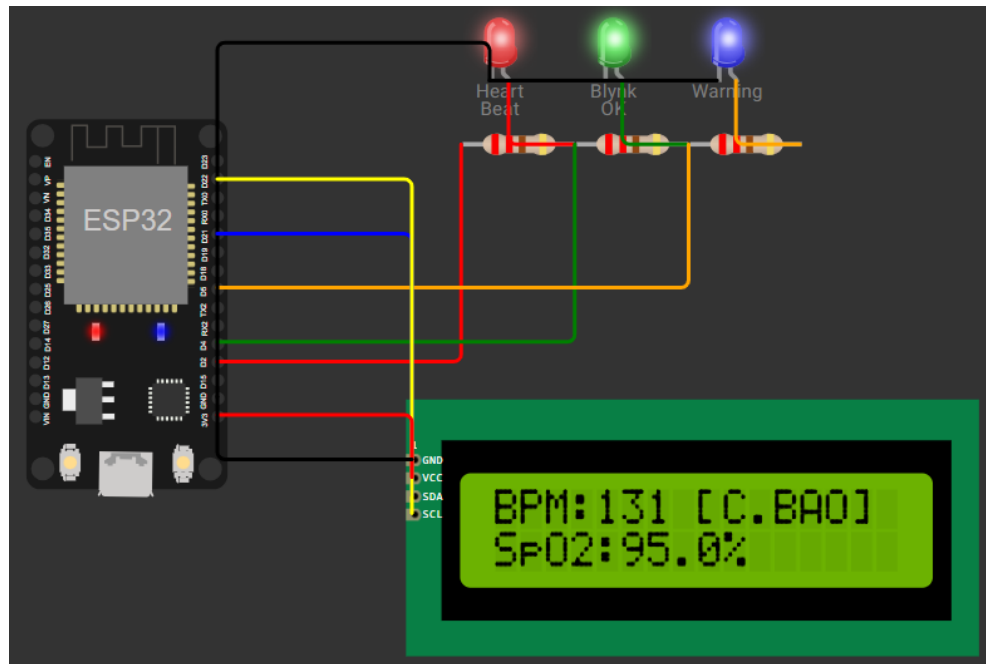


Hình 8: Kết quả mô phỏng Wokwi ở trạng thái bình thường

3.2.3. *Trạng thái cảnh báo*

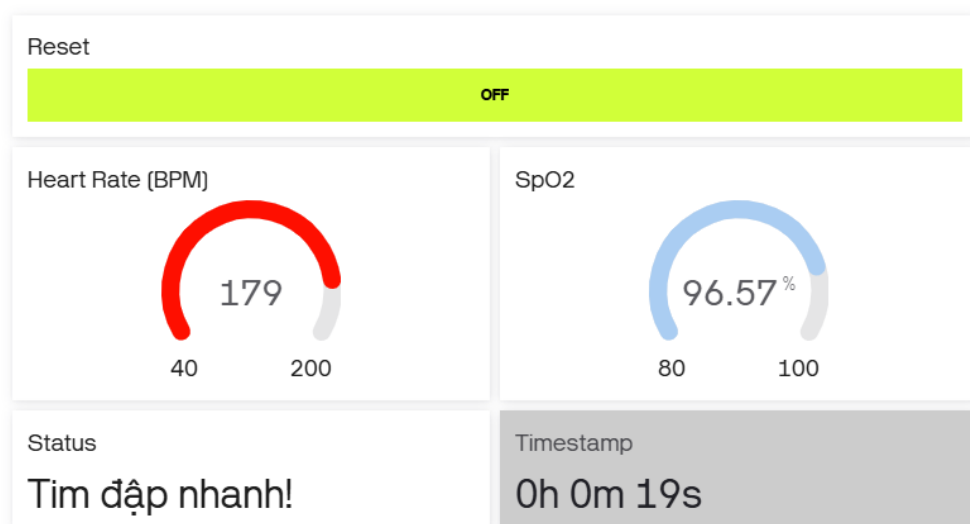
Khi chỉ số BPM hoặc SpO2 vượt ra ngoài ngưỡng an toàn (có thể do drift sinh lý mô phỏng hoặc nhiễu chuyển động), hệ thống sẽ lập tức phản hồi: trạng thái trên giao diện Blynk chuyển sang mức cảnh báo tương ứng (ví dụ:

Tim đập nhanh! hoặc *Thiếu SPO2!*), đồng thời LED vàng (Warning) được kích hoạt sáng lên để báo động trực quan.



Hình 9: Kết quả mô phỏng Wokwi khi kích hoạt cảnh báo

3.3. Kết quả trên Blynk Dashboard



Hình 10: Blynk Dashboard hiển thị dữ liệu nhịp tim theo thời gian thực

Dữ liệu được gửi lên đám mây Blynk định kỳ mỗi 5 giây, bao gồm các thông số: nhịp tim (Virtual Pin V0), SpO2 (V1), trạng thái sức khỏe (V2) và thời gian hoạt động của hệ thống (V3). Bên cạnh đó, khi người dùng nhấn nút Reset trên Dashboard, vi điều khiển ESP32 sẽ nhận được lệnh thông qua hàm callback `BLYNK_WRITE(V5)` và tiến hành đặt lại màn hình hiển thị cục bộ.

3.4. Phân tích và đánh giá

3.4.1. Phân tích kết quả theo các tình huống

Bảng 11: Kết quả kiểm thử các tình huống

Tình huống	BPM	SpO2	Status	LED cảnh báo
Bình thường	65 – 95	96 – 99%	Normal	Tắt
Nhịp chậm	< 60	≥ 95%	Bradycardia	Sáng
Nhịp nhanh	> 100	≥ 95%	Tachycardia	Sáng
SpO2 thấp	Bình thường	< 95%	LOW SPO2!	Sáng
Motion artifact	Dao động	Dao động	Có thể cảnh báo	Có thể sáng
Mất kết nối	Bình thường	Bình thường	Normal	LED xanh tắt

3.4.2. Đánh giá ưu điểm và hạn chế

Ưu điểm đạt được:

- **Mô phỏng vật lý trung thực:** Thay vì sinh số ngẫu nhiên đơn giản, module MAX30100_Sim.h tái tạo đúng pipeline PPG, đảm bảo giá trị BPM và SpO2 sinh lý hợp lệ và dao động tự nhiên.
- **Warm-up giống cảm biến thật:** Người dùng thấy rõ quá trình khởi động, không bị nhầm lẫn với lỗi hệ thống.
- **Thiết kế module hóa:** API tương thích giúp dễ dàng chuyển sang phần cứng thực mà không cần sửa logic chính trong main.cpp.
- **Giám sát đa kênh:** Dữ liệu được hiển thị đồng thời tại thiết bị (LCD, LED) và từ xa (Blynk dashboard).

Hạn chế còn tồn tại:

- **Giới hạn của mô phỏng:** Dữ liệu không phản ánh được các biến động sinh lý thực tế phức tạp như bệnh lý, vận động mạnh hay thay đổi tư thế.
- **Không lưu lịch sử:** Dữ liệu chỉ hiển thị theo thời gian thực (realtime), chưa có cơ sở dữ liệu để phân tích xu hướng dài hạn.
- **Tiêu thụ điện năng:** Việc duy trì kết nối WiFi liên tục tiêu tốn ~240mA, chưa tối ưu cho các thiết bị đeo tay chạy bằng pin dung lượng nhỏ.
- **Giới hạn nền tảng:** Tài khoản Blynk miễn phí bị giới hạn số lượng datastream và tần suất cập nhật dữ liệu.

3.5. Hướng phát triển

1. **Triển khai phần cứng thực:** Thay module mô phỏng bằng thư viện MAX30100lib chính thức và lắp ráp mạch thật để đo nhịp tim người dùng thực tế.

2. **Lưu trữ và phân tích dữ liệu:** Tích hợp Firebase Firestore hoặc ThingSpeak để lưu trữ lịch sử, vẽ đồ thị xu hướng theo ngày/tuần.
3. **Cảnh báo tự động:** Bổ sung bot Telegram để gửi tin nhắn khẩn cấp tức thì khi phát hiện nhịp tim hoặc SpO2 ở ngưỡng nguy hiểm.
4. **Tối ưu năng lượng:** Áp dụng chế độ Deep Sleep của ESP32, chỉ thức dậy mỗi 30 giây để đo và gửi dữ liệu, giúp kéo dài thời lượng pin.
5. **Tích hợp Trí tuệ nhân tạo (AI):** Ứng dụng TinyML trên ESP32 để nhận diện các rối loạn nhịp tim (như rung nhĩ - AFib) trực tiếp từ dạng sóng PPG, nâng cao giá trị y tế của thiết bị.
6. **Mở rộng cảm biến:** Kết hợp thêm cảm biến DS18B20 để đo thân nhiệt và MPU6050 để đếm bước chân, hướng tới một hệ sinh thái theo dõi sức khỏe toàn diện.

KẾT LUẬN

Qua quá trình nghiên cứu và thực hiện đề tài, nhóm đã đạt được các kết quả nổi bật sau:

Về mặt lý thuyết, nhóm đã nắm vững nguyên lý đo nhịp tim và SpO2 bằng phương pháp quang dung lượng (PPG), hiểu sâu cơ chế hoạt động của cảm biến MAX30100, quy trình xử lý tín hiệu từ dữ liệu thô đến kết quả sinh lý, cũng như kiến trúc hệ thống IoT từ lớp thiết bị biên đến nền tảng đám mây.

Về mặt thực hành, nhóm đã xây dựng hoàn chỉnh hệ thống bao gồm: module mô phỏng vật lý MAX30100_Sim.h với đầy đủ tính năng warm-up, trôi sinh lý (drift) và nhiễu chuyển động (motion artifact); sơ đồ mạch trên Wokwi kết nối ESP32, LCD 1602 và hệ thống LED chỉ thị; phần mềm nhúng hoàn chỉnh gửi dữ liệu qua WiFi; và dashboard Blynk để hiển thị thông số sức khỏe theo thời gian thực.

Bài học kinh nghiệm: Điểm giá trị nhất mà đề tài mang lại là nhận thức rõ sự khác biệt giữa *mô phỏng* và *phản ứng thực*. Mặc dù môi trường Wokwi không hỗ trợ sẵn MAX30100, nhưng thay vì sử dụng hàm nội suy số ngẫu nhiên đơn giản, nhóm đã chủ động tái tạo toàn bộ pipeline vật lý — từ việc sinh tín hiệu theo phương trình PPG, tính toán ngược thành phần AC/DC, cho đến việc áp dụng định luật Beer-Lambert. Điều này giúp kết quả mô phỏng mang lại giá trị giáo dục thực tiễn cao.

Đồng thời, hệ thống được thiết kế theo tư duy module hóa vững chắc, tách biệt hoàn toàn lớp trừu tượng cảm biến khỏi logic luồng xử lý chính. Nhờ đó, việc chuyển đổi thiết kế này sang môi trường phần cứng thực tế trong tương lai sẽ diễn ra dễ dàng với những thay đổi tối thiểu ở mức mã nguồn.

TÀI LIỆU THAM KHẢO

Tiếng Việt:

- [1] Nguyễn Đình Phú (2020). *Lập trình vi điều khiển ESP32 với Arduino*. NXB Đại học Quốc gia TP.HCM.
- [2] Trần Văn Hùng (2021). *Internet of Things – Kết nối vạn vật*. NXB Bách Khoa Hà Nội.

Tiếng Anh:

- [3] Maxim Integrated (2014). *MAX30100 Pulse Oximeter and Heart-Rate Sensor IC for Wearable Health – Datasheet*. Rev 2. <https://www.analog.com/media/en/technical-documentation/data-sheets/max30100.pdf>.
- [4] Espressif Systems (2023). *ESP32 Series Datasheet*. Version 4.4. https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.
- [5] Blynk Inc. (2024). *Blynk Documentation – ESP32 with Arduino*. <https://docs.blynk.io/en/getting-started/what-do-i-need-to-blynk>. Truy cập ngày 01/06/2025.
- [6] Wokwi (2024). *Wokwi for VS Code – Simulation Documentation*. <https://docs.wokwi.com/vscode/getting-started>.
- [7] Allen, J. (2007). Photoplethysmography and its application in clinical physiological measurement. *Physiological Measurement*, 28(3), R1–R39.
- [8] Mendelson, Y., & Ochs, B. D. (1988). Noninvasive pulse oximetry utilizing skin reflectance photoplethysmography. *IEEE Transactions on Biomedical Engineering*, 35(10), 798–805.